FITNESS TRACKER

MERN STACK

PRESENTED BY : MS AMNA
ADSE I
STUDENT 1525122

TABLE OF CONTENTS

Fitness Tracker

Aptech
Unleash your potential

CERTIFICATE

OF ACHIEVEMENT

THIS CERTIFICATE IS AWARDED TO

Ms Amna

FOR COMPLETING SEMESTER END PROJECT OF 5TH SEM,
PRVIDED BY THE E-PROJECT TEAM

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Aptech Institute and the eProject Team for providing the opportunity to work on this real-life Fitness Tracker application. This project helped me apply practical concepts of MongoDB, Express.js, React.js, and Node.js in a structured manner. I am thankful to my mentor, Ms Kiran Shakeel for her guidance and support throughout the project as she helped me achieve the completion of the project in due time without any hurdles or difficulties. She helped us through the way and cleared our confusions and helped boost our ideas.

PROBLEM STATEMENT

Problem statement

Maintaining a consistent fitness routine is difficult due to the lack of structured and centralized tracking systems. Many individuals rely on manual methods or multiple disconnected applications, making it hard to monitor workouts, nutrition, and overall progress effectively. Additionally, some existing solutions lack proper data visualization or secure user-specific access. Therefore, there is a need for a secure, user-friendly web application that allows users to track workouts, monitor nutrition, and visualize their fitness progress in an organized and efficient manner.

Proposed Solution

To address the identified problem, a full-stack Fitness Tracker web application is proposed. The system provides a centralized platform where users can securely register, log in, and manage their fitness-related data.

The proposed solution includes:

- Secure authentication using JSON Web Tokens (JWT)
- Workout logging and activity tracking
- Nutrition monitoring including calories and macronutrients
- Interactive dashboard with graphical progress visualization
- Responsive and modern user interface for improved user experience

The application integrates frontend and backend technologies to ensure secure data handling, structured storage, and efficient performance. By combining workout tracking, nutritional monitoring, and visual progress analytics into a single system, the solution promotes accountability, consistency, and data-driven fitness management.

Application Purpose :

The main purpose of this application is to help users:

- Track their daily workouts
- Monitor calorie intake and macronutrients
- Visualize weekly fitness progress
- Maintain consistency through data-driven insights

It provides a centralized platform where users can manage their fitness journey in a structured and secure way.

Keay features :

- Secure user authentication using JWT
- Workout logging and activity tracking
- Nutrition tracking (calories, protein, carbs, fats)
- Interactive dashboard with progress visualization
- Protected routes and secure API access
- Responsive modern UI design

Technologies Used :

Frontend:

- React.js – for building the user interface
- Tailwind CSS – for modern and responsive styling
- Framer Motion – for animations
- Axios – for API communication
- React Router – for routing between pages

Backend:

- Node.js – runtime environment
- Express.js – backend framework
- MongoDB – NoSQL database
- JWT (JSON Web Token) – for authentication
- Mongoose – for database modeling

Target users :

The target users of this application are:

- Individuals who want to track their fitness progress
- Gym members and athletes
- Health-conscious users monitoring calories and nutrition
- Beginners starting their fitness journey

It is designed for anyone who wants a simple but powerful fitness tracking solution.

PROJECT REQUIREMENTS

Requirement	Status
User Management	✓
Fitness Tracking	✓
Dashboard	✓
Data Visualization	✓
User Profile	✓
Search and Filtering	✓
Mobile Compatibility	✓

PROJECT REQUIREMENTS

Requirement	Status
Search and Filtering	✓
User Preferences	✓
Reporting and Export	✓
Notifications and Alerts	✓
Search and Filtering	✓

ABOUT PROJECT

What the project is :

The Fitness Tracker is a full-stack web-based application developed to provide users with a structured and secure platform for monitoring and managing their fitness activities. The system enables users to register and authenticate securely, log workouts, track nutritional intake, and analyze progress through an interactive dashboard interface.

The application is developed using modern web technologies, with React.js for the frontend user interface and Node.js with Express.js for backend services. MongoDB is used as the database for storing user and activity data, while JSON Web Tokens (JWT) are implemented to ensure secure authentication and protected route access.

The primary objective of this project is to promote consistency and accountability in personal fitness management by integrating workout tracking, calorie monitoring, and performance visualization within a centralized system. The dashboard provides users with graphical representations of their weekly progress, enabling data-driven decision-making and improved goal tracking.

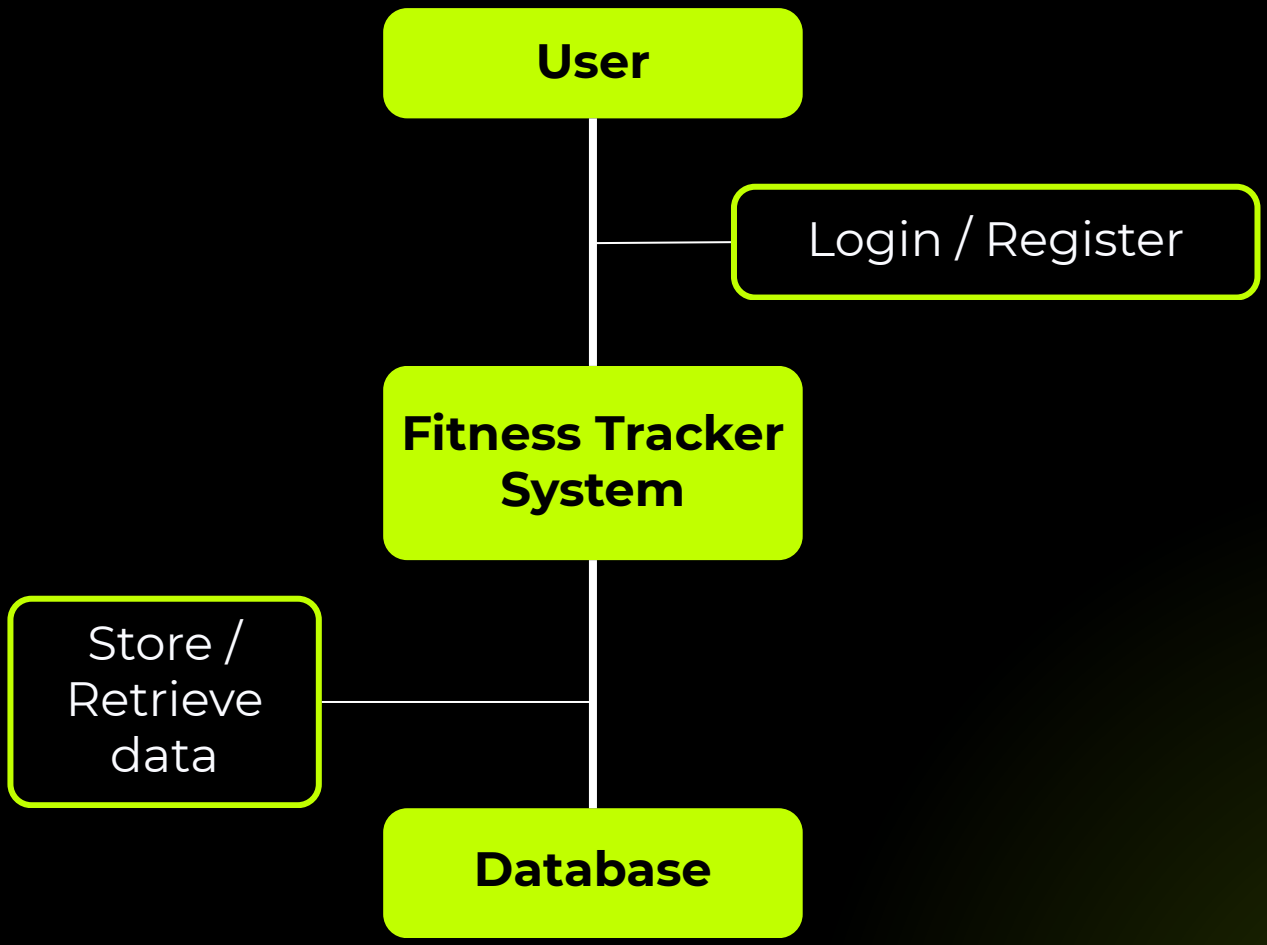
This project demonstrates the implementation of full-stack development principles, RESTful API integration, secure authentication mechanisms, and responsive user interface design.

WORK ANALYSES

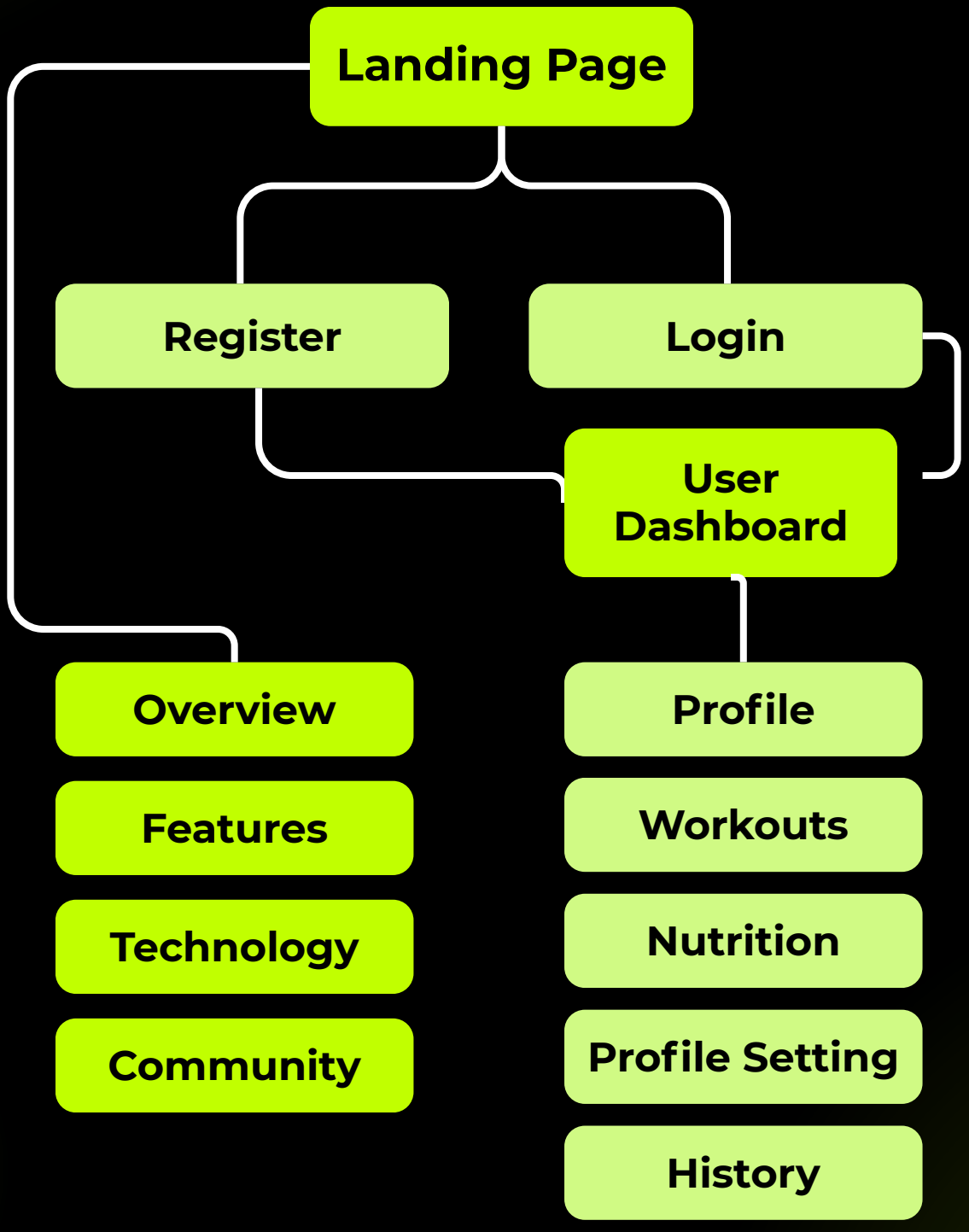
Phases	Days
Analyses	22 Jan 2026
Planning	24 Jan 2026
Designing	24 Jan 2026
Testing	2 Feb 2026
Project wrap	16 Feb 2026
Presentation	20 Feb 2026

DATA FLOW DIAGRAM

Level (0) DFD



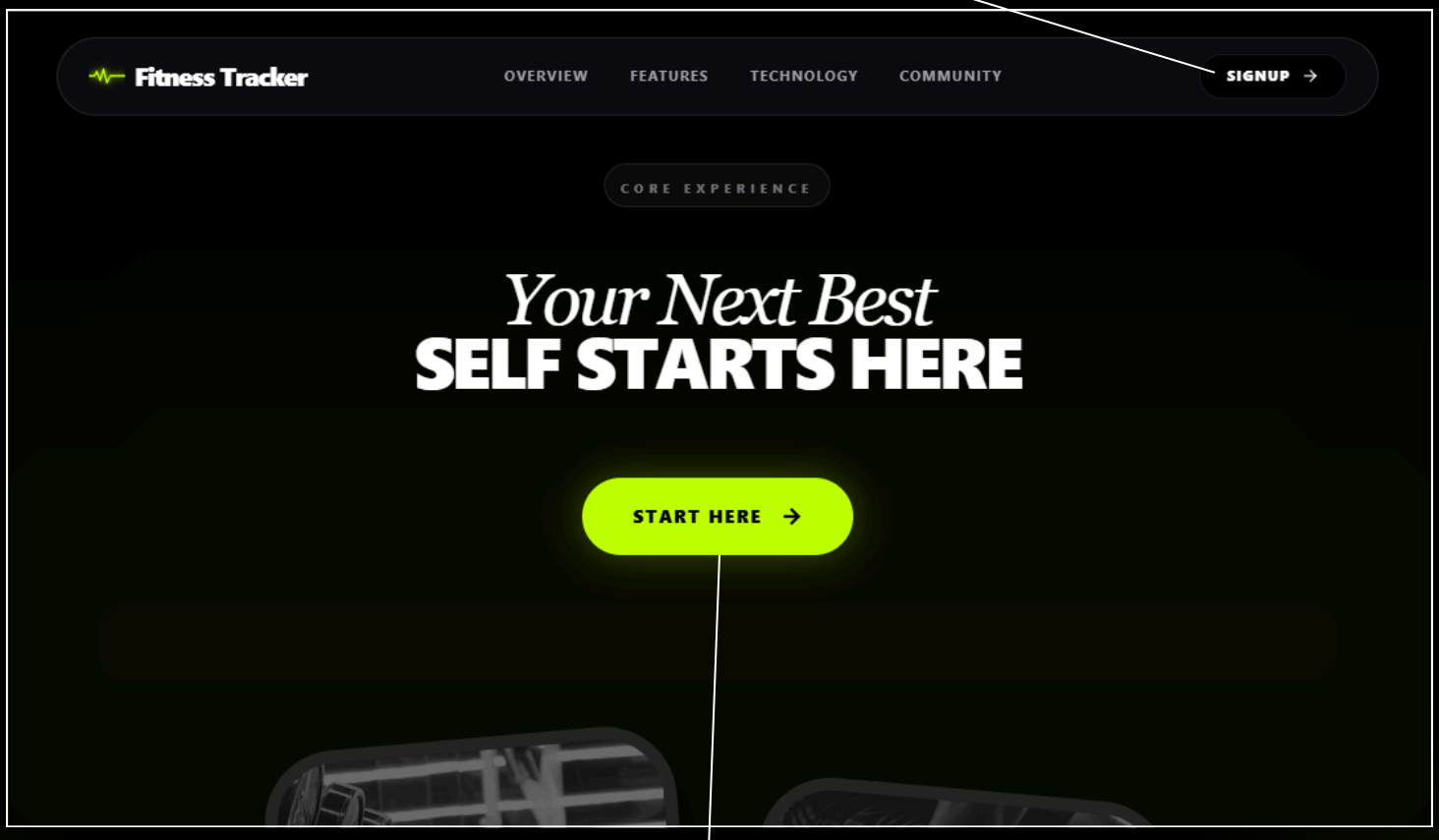
USER FLOW DIAGRAM



SCREENSHOTS

Landing page

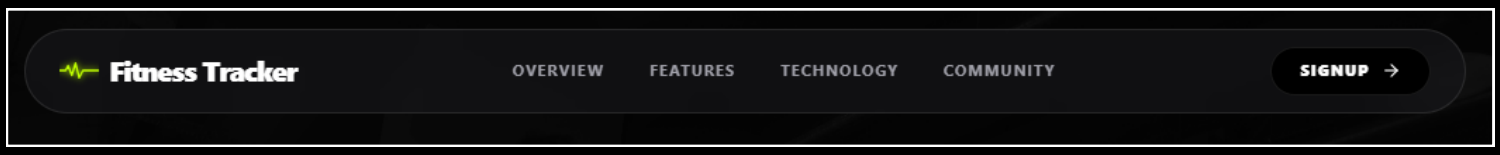
Logged out users can login or register after entering the application



'Start Here' button leading to Login page

SCREENSHOTS

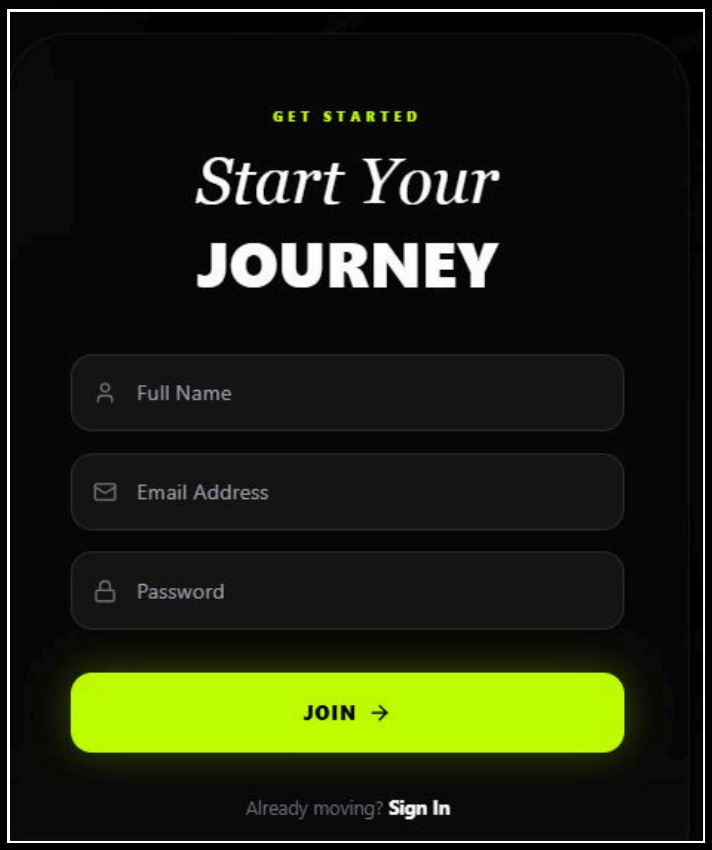
Navbar

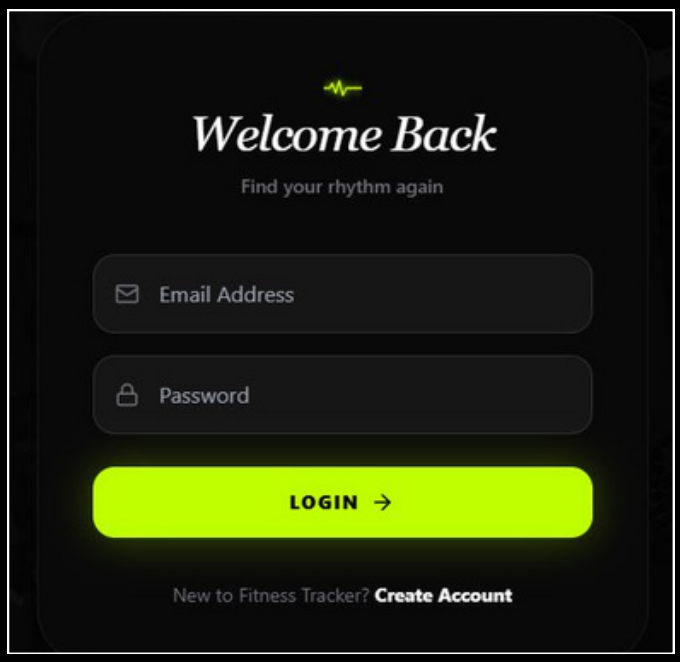


Footer

SCREENSHOTS

1) User Management System





- Secure user registration and login (JWT Authentication)
- Personalized user profile with image upload
- Update profile details
- Secure password encryption (bcrypt)

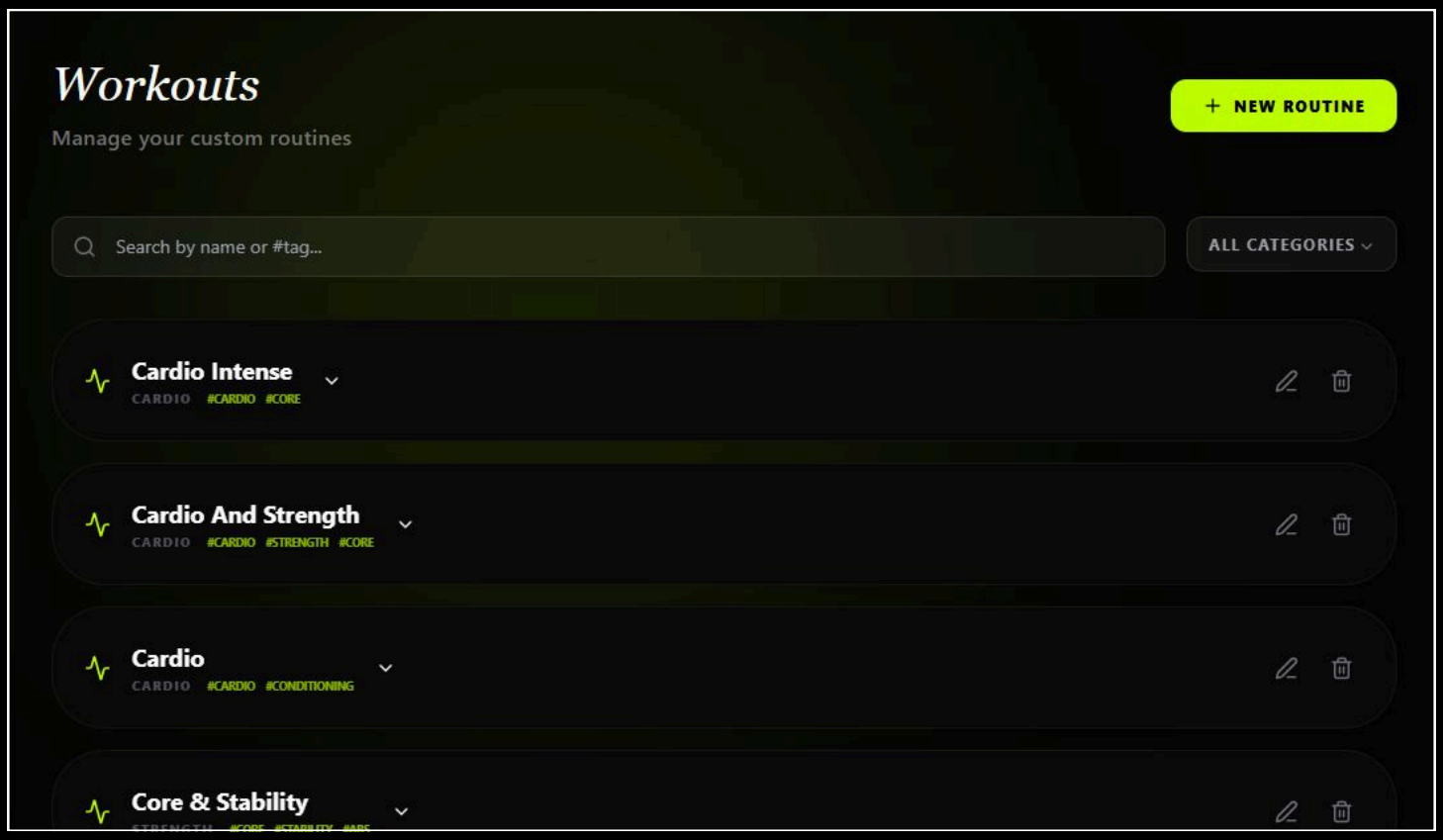
SCREENSHOTS

Side navigation

- Dashboard (User information and workout charts and nutrition log)
- Workout CRUD
- Adding new meals to count Nutritions
- User's profile setting (email, username, profile picture, bio)
- Workout History saved as logs to let the users view their workout history

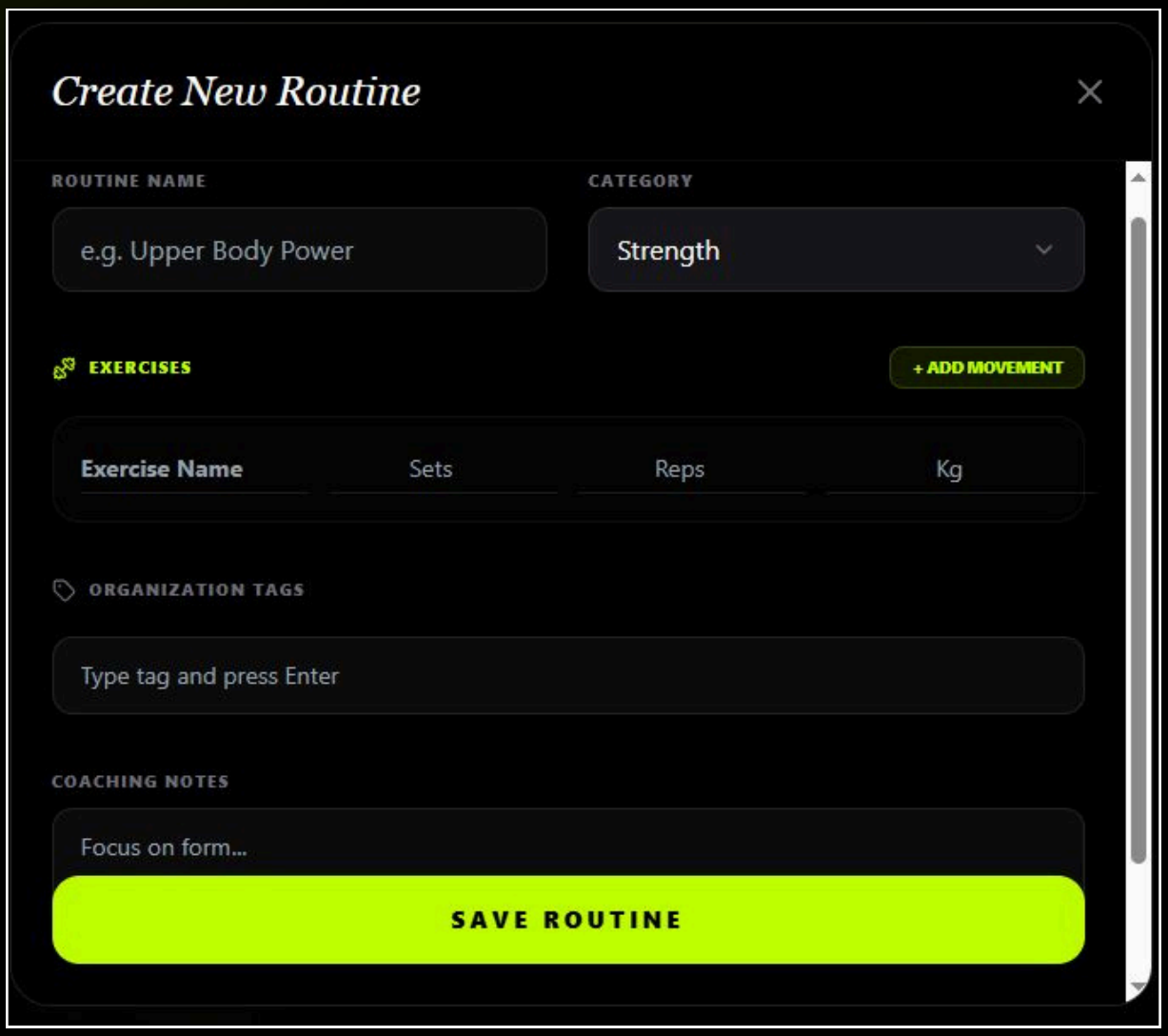
SCREENSHOTS

2) Workout Management Module



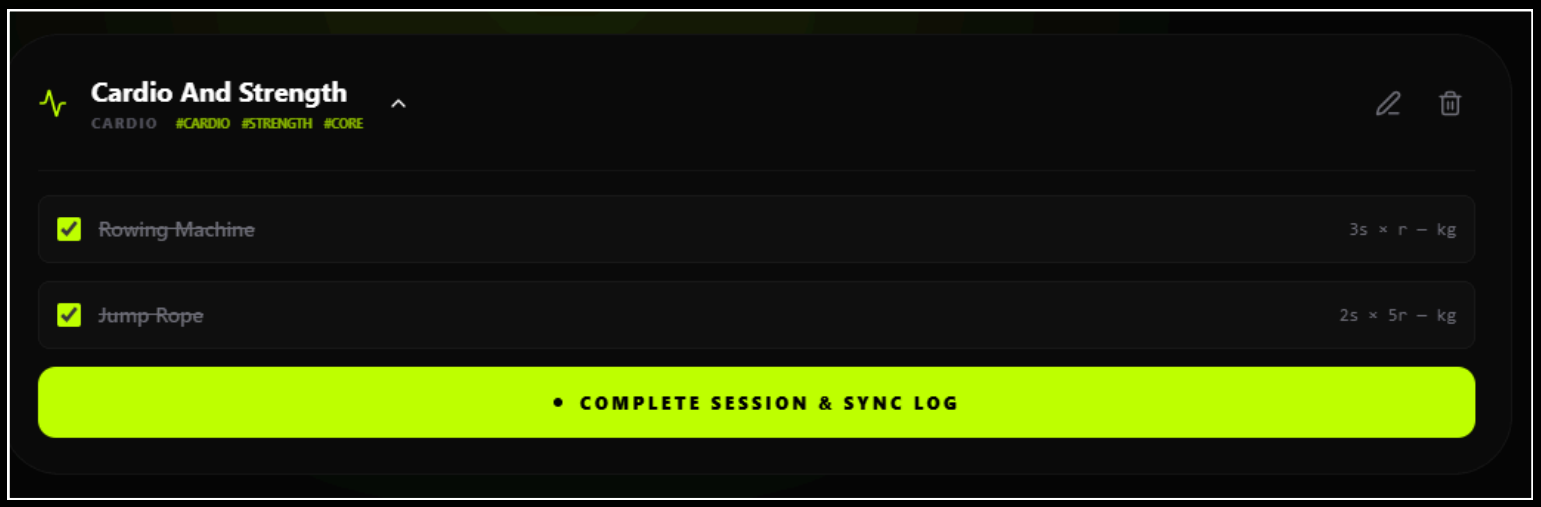
- Create new workout routines
- Edit existing workouts
- Delete workouts
- Checklist-style workout list view

SCREENSHOTS



- Categorization (Strength / Cardio / Custom)
- Tag-based organization
- Each workout includes:
- Exercise name
- Sets
- Reps
- Weight
- Notes

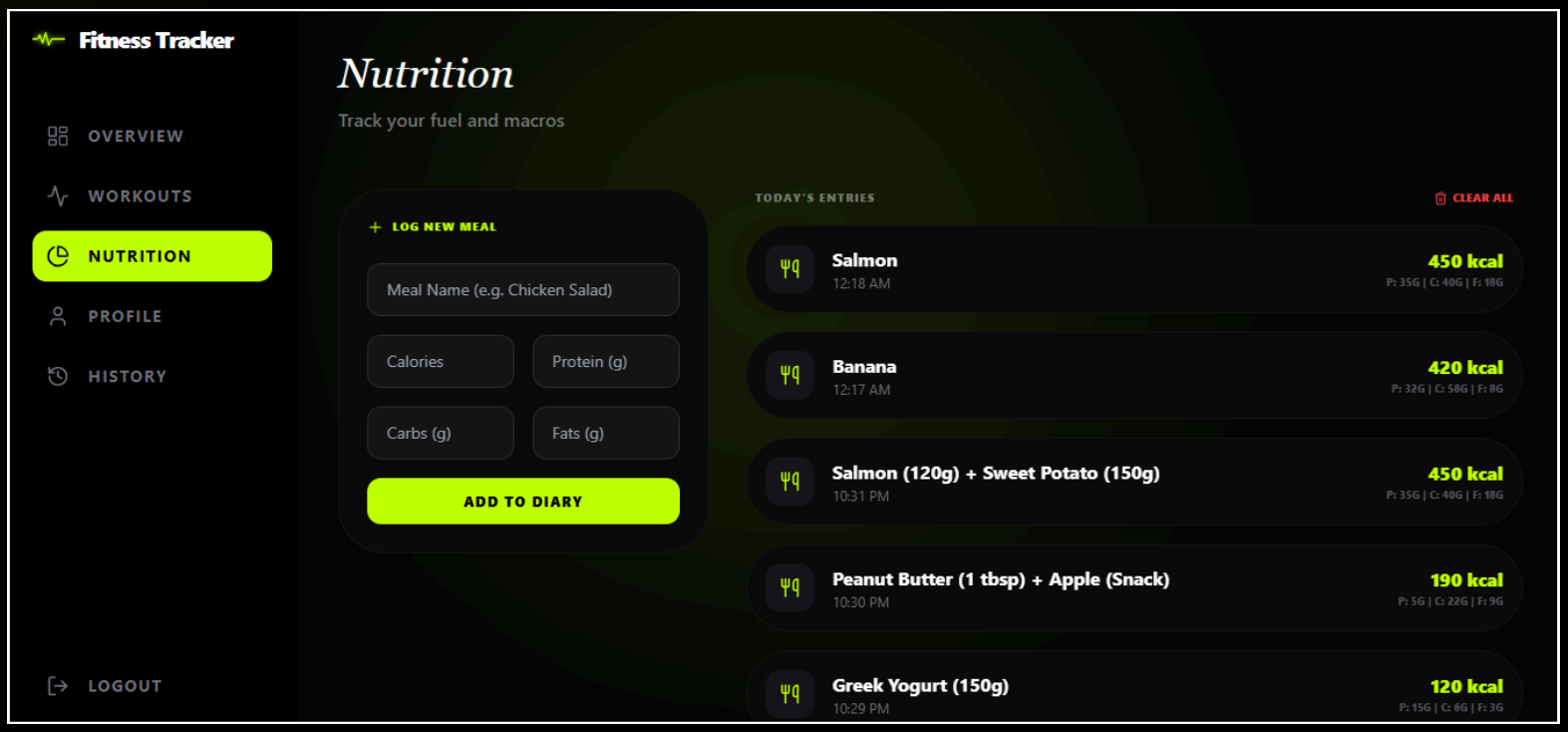
SCREENSHOTS



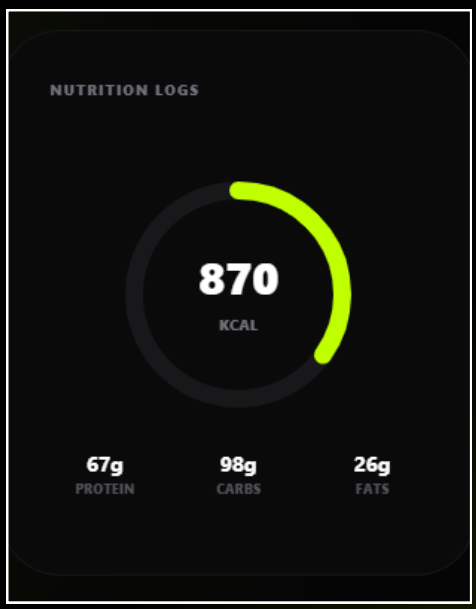
- Checklist-style workout list view
- Ending each workout session with single button to log in History

SCREENSHOTS

3) Nutrition Tracking System

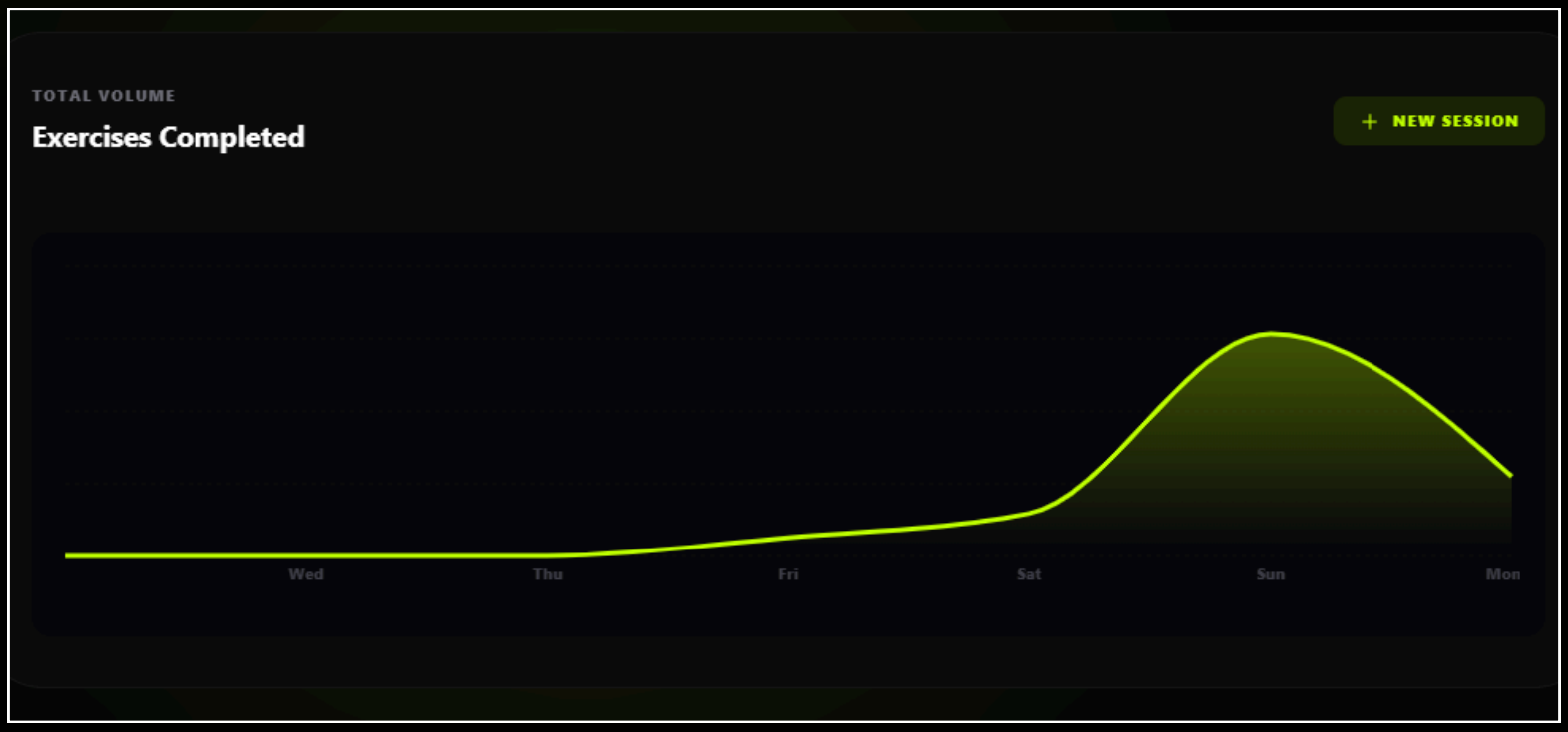


- Log daily meals (Breakfast, Lunch, Dinner, Snacks)
- Track calories and macronutrients
- Edit and delete nutrition entries
- Daily nutrition summary

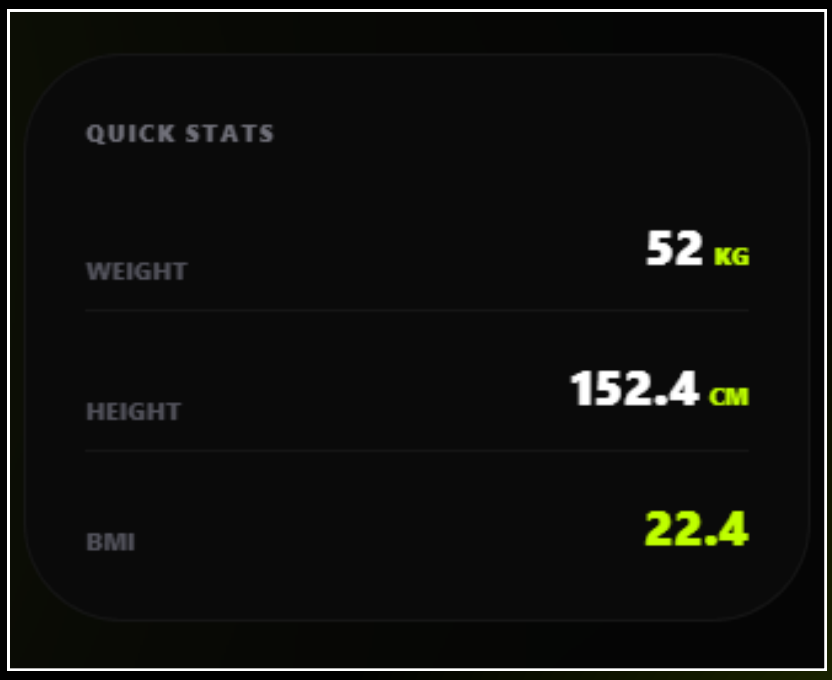


SCREENSHOTS

4) Progress Tracking & Analytics

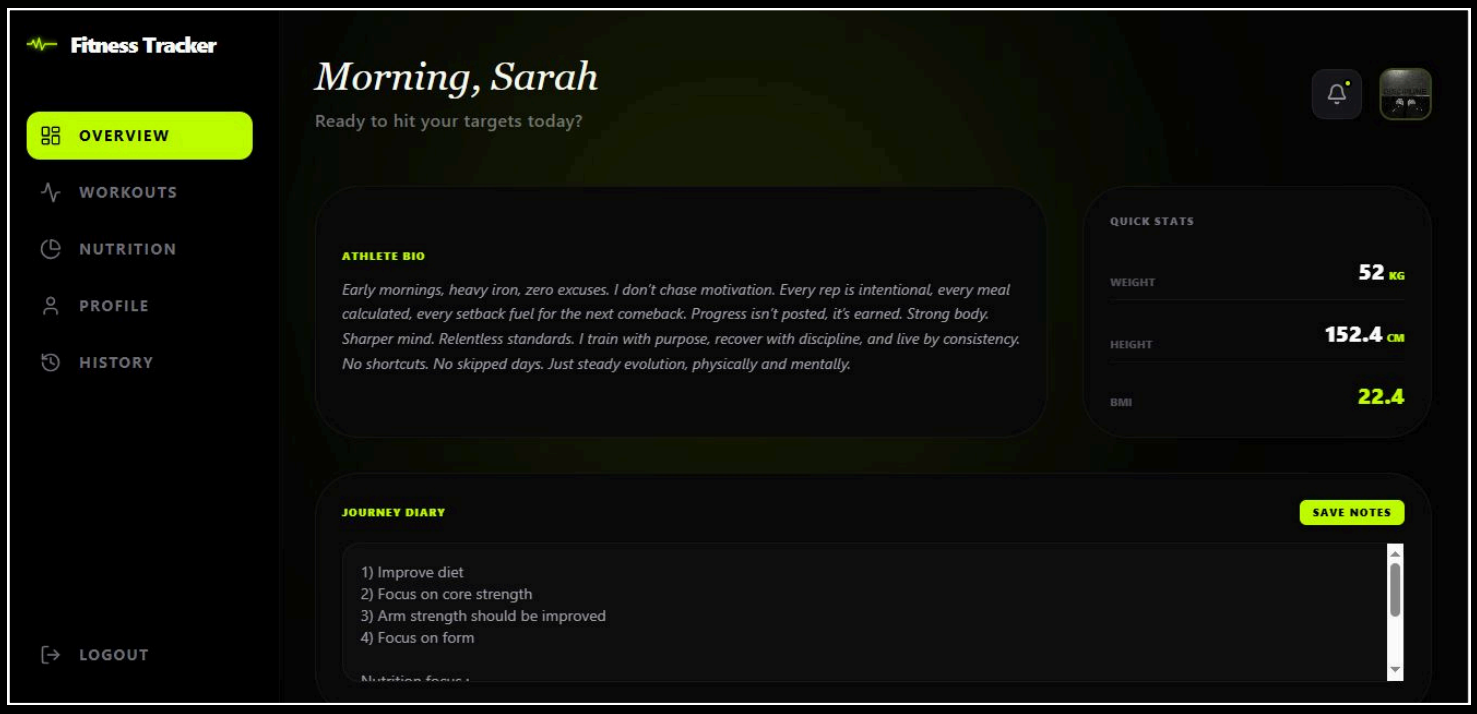


- Record weight and body measurements
- Track performance metrics
- Visual charts and graphs
- Workout analytics
- Nutrition analytics



SCREENSHOTS

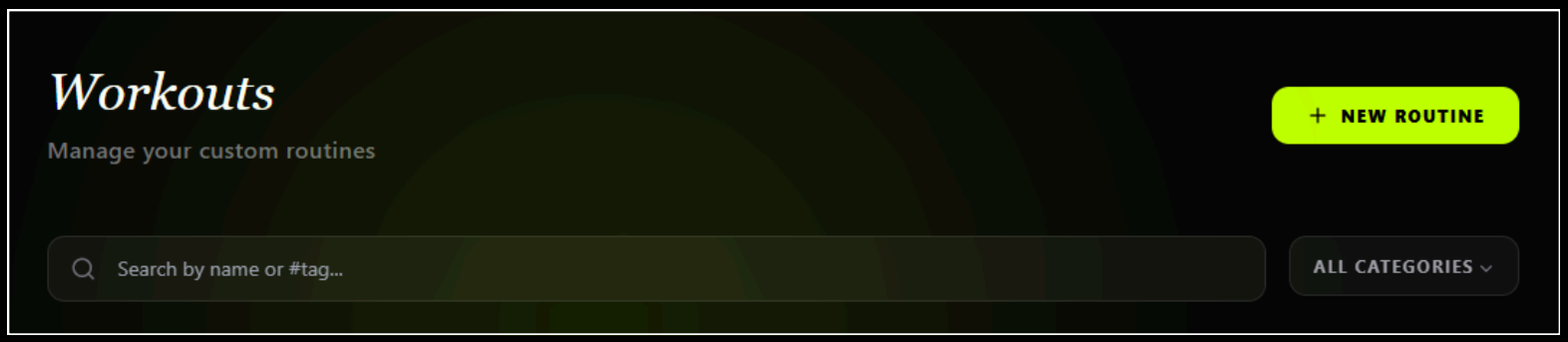
5) Personalized Dashboard



- Overview of recent workouts
- Nutrition summary
- Progress visualization
- Quick access “New Workout” button
- Activity notifications

SCREENSHOTS

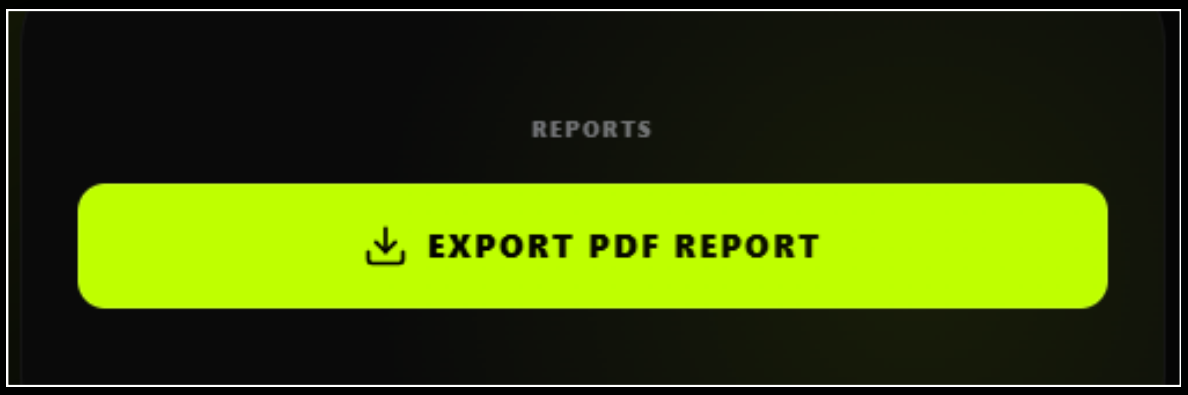
6) Search & Filter System



- Search workouts by name
- Filter by category
- Sort by date or type

SCREENSHOTS

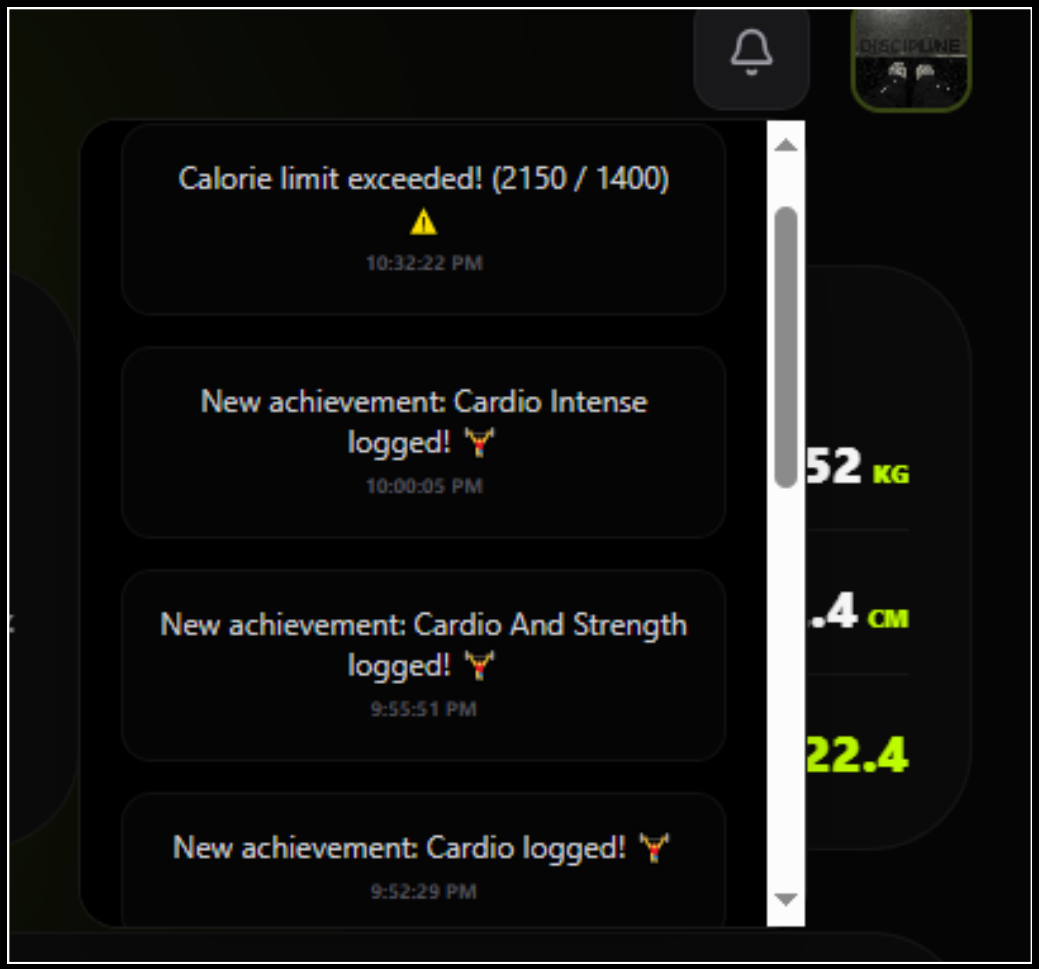
7) Data Export & Reporting



- Export progress reports (PDF / CSV)
- Download workout history
- Nutrition data reports

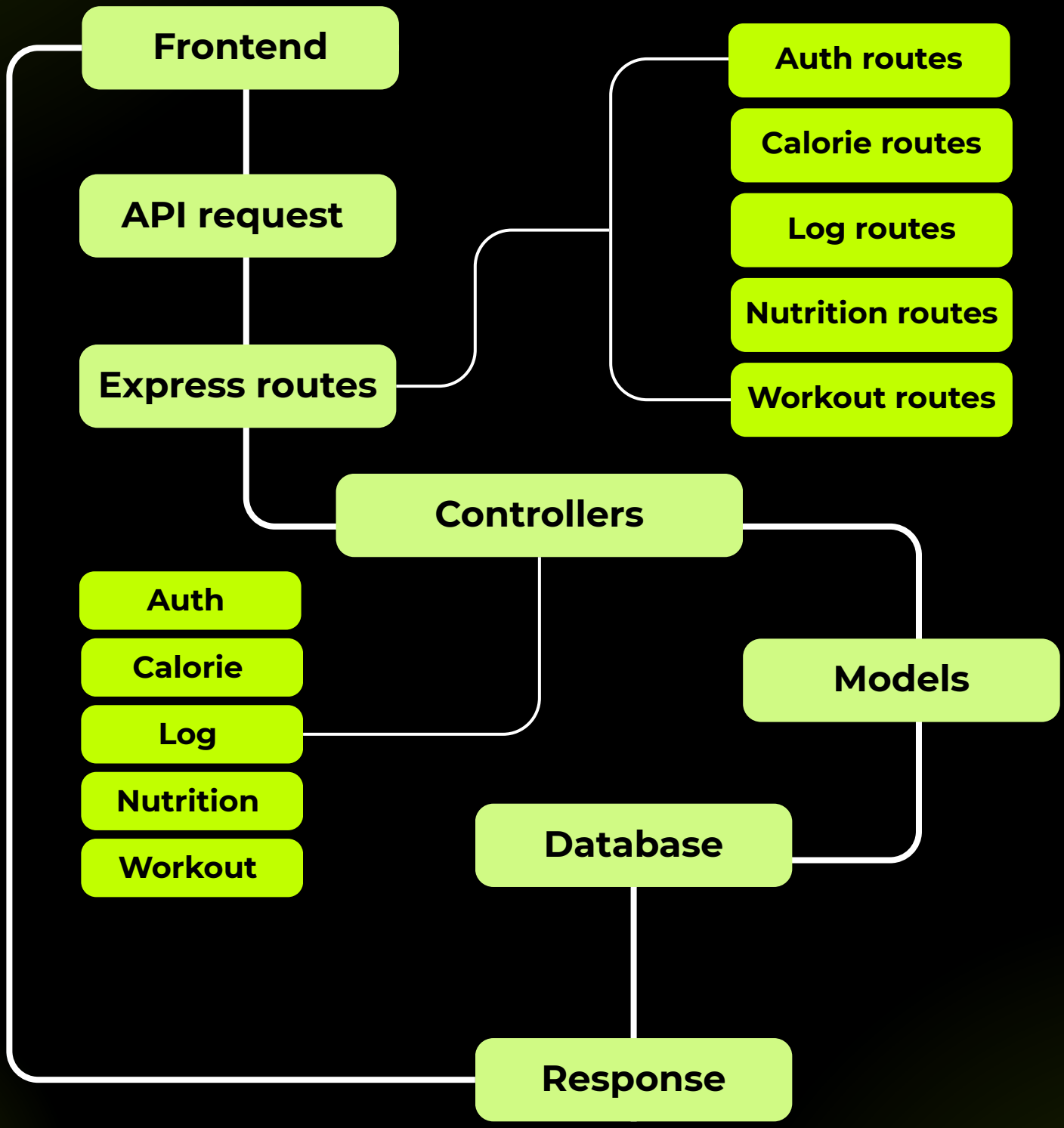
SCREENSHOTS

8) Notifications & Reminders

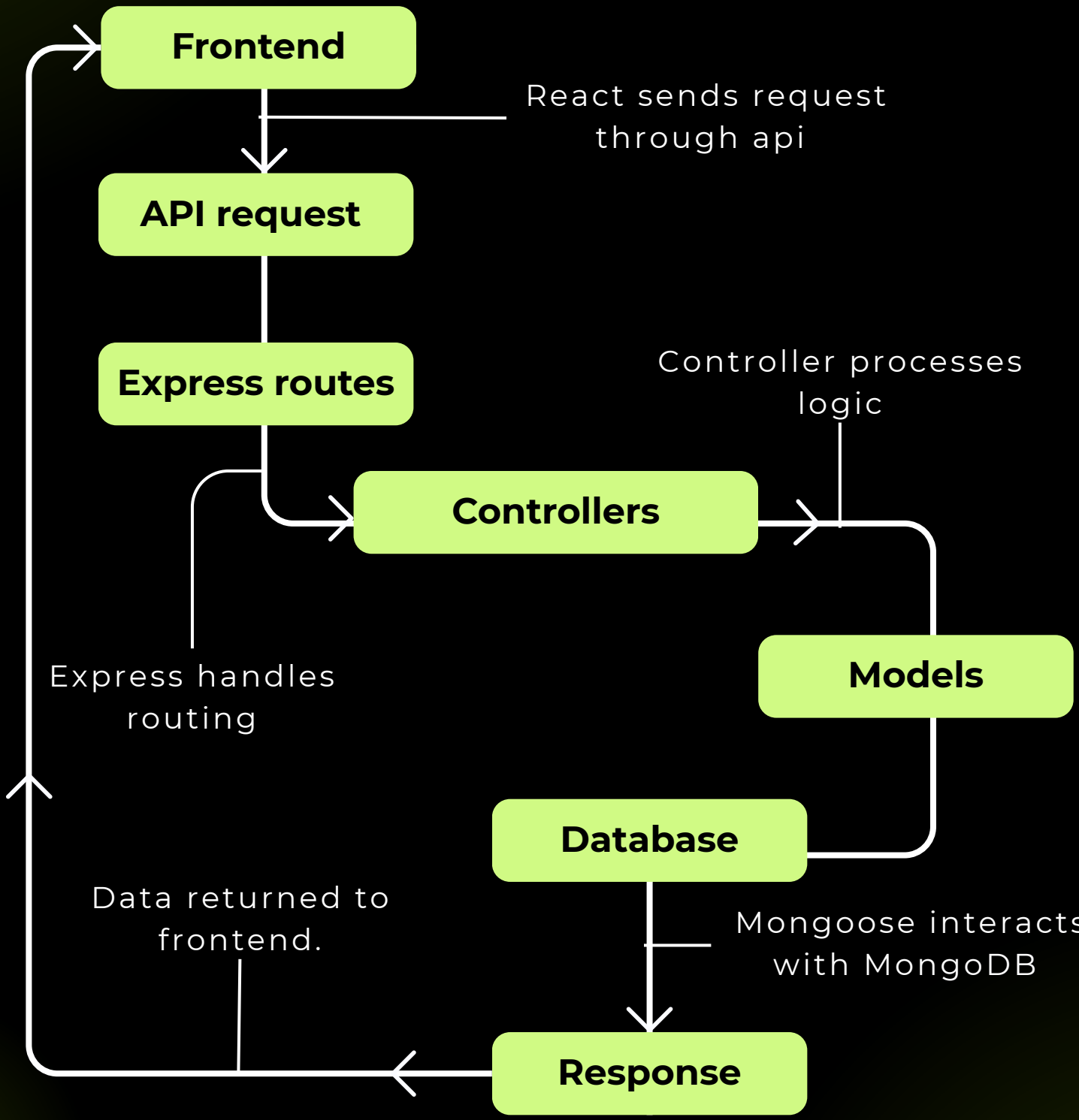


- Workout reminders
- Meal alerts
- Goal achievement notifications

STRUCTURE DIAGRAM



PROCESS DIAGRAM



ER DIAGRAM

Users

_id (ObjectId)

name (String)

email (String, Unique)

password (String, Hashed)

profilePicture (String)

createdAt (Date)

updatedAt (Date)

Nutritions

Foreign Key: user → Users._id

- _id (ObjectId)
- user (ObjectId)
- foodName (String)
- calories (Number)
- protein (Number)
- carbs (Number)
- fats (Number)
- createdAt (Date)
- updatedAt (Date)

ER DIAGRAM

Workouts

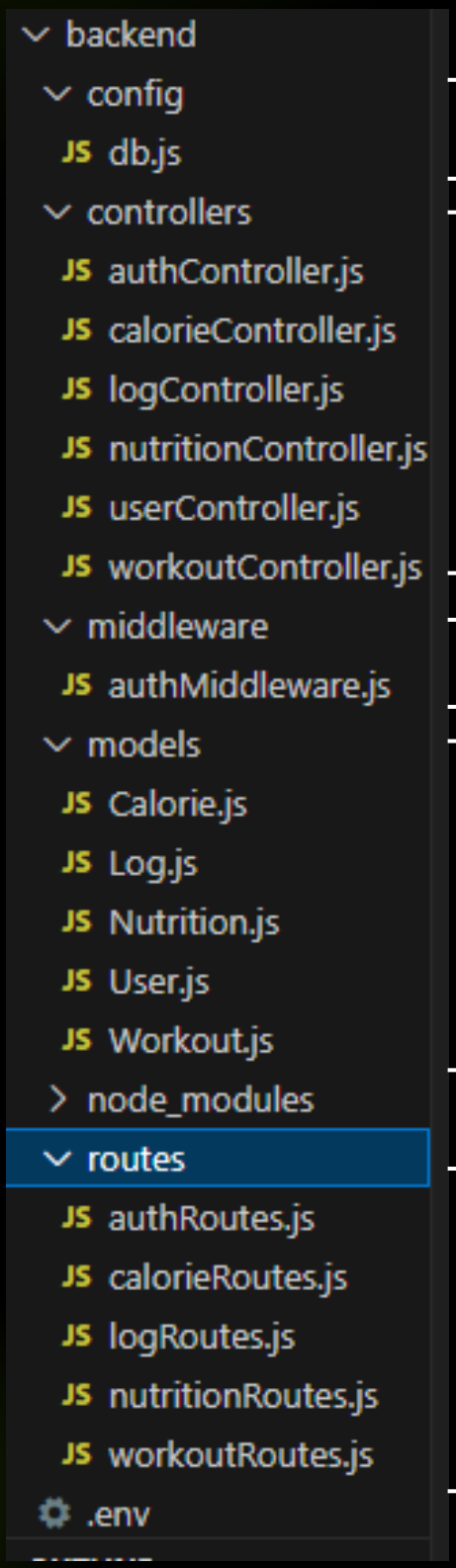
Foreign Key: user → Users._id

- _id (ObjectId)
- user (ObjectId)
- name (String)
- category (String)
- exercises (Array)
- tags (Array)
- createdAt (Date)
- updatedAt (Date)

Logs

- _id (ObjectId)
- user (ObjectId)
- workoutName (String)
- category (String)
- exercises (Array)
- completedAt (Date)
- createdAt (Date)
- updatedAt (Date)

FOLDER STRUCTURE



Database connectivity

CRUD api for Workout, Nutritions, Calories and User Auth

User authentication using JWT in middleware for ProtectedRoutes

Models defining data structure to store in MongoDB database.

Handling all Api routes

CODE SNIPPETS

Database connectivity in backend

db.js

backend > config > db.js > ...

```
1   const mongoose = require('mongoose');
2
3   const connectDB = async () => {
4     try {
5       const conn = await mongoose.connect(process.env.MONGO_URI);
6       console.log(`MongoDB Connected: ${conn.connection.host}`);
7     } catch (error) {
8       console.error(`Error: ${error.message}`);
9       process.exit(1);
10    }
11  };
12
13  module.exports = connectDB;
```

Controller :

Auth controller handling user registration, login, update user data and saving the information without ending session.

```
// Register user

const registerUser = async (req, res) => {
  const { name, email, password } = req.body;

  const userExists = await User.findOne({ email });

  if (userExists) {
    return res.status(400).json({ message: 'User already exists' });
  }

  const user = await User.create({ name, email, password });

  if (user) {
    res.status(201).json({
      _id: user._id,
      name: user.name,
      email: user.email,
      token: generateToken(user._id),
    });
  } else {
    res.status(400).json({ message: 'Invalid user data' });
  }
};
```

CODE SNIPPETS

Handling Workout CRUD in workoutController

authController.js | workoutController.js

backend > controllers > workoutController.js > ...

```
44  const getWorkouts = async (req, res) => {
46      const workouts = await Workout.find({ user: req.user._id }).sort({ createdAt: -1 });
47      res.status(200).json(workouts);
48    } catch (error) {
49      res.status(500).json({ message: 'Server Error' });
50    }
51  };
52
53  // @desc    Delete a workout
54  // @route   DELETE /api/workouts/:id
55  const deleteWorkout = async (req, res) => {
56    const workout = await Workout.findById(req.params.id);
57    if (workout && workout.user.toString() === req.user._id.toString()) {
58      await workout.deleteOne();
59      res.json({ message: 'Workout removed' });
60    } else {
61      res.status(404).json({ message: 'Workout not found' });
62    }
63  };
```

Model:

Log.js storing Workout log as History

backend > models > Log.js > ...

```
1  const mongoose = require('mongoose');
2
3  const logSchema = mongoose.Schema({
4    user: { type: mongoose.Schema.Types.ObjectId, required: true, ref: 'User' },
5    workoutName: { type: String, required: true },
6    category: { type: String },
7    exercises: [
8      {
9        exerciseName: String,
10       sets: Number,
11       reps: Number,
12       weight: Number,
13     }
14   ],
15   completedAt: { type: Date, default: Date.now }
16 }, { timestamps: true });
17
18 module.exports = mongoose.model('Log', logSchema);
```

CODE SNIPPETS

User updating workout data

```
// Workout update

const updateWorkout = async (req, res) => {
  const { name, category, exercises, notes, tags } = req.body;
  const workout = await Workout.findById(req.params.id);

  if (workout && workout.user.toString() === req.user._id.toString()) {
    workout.name = name || workout.name;
    workout.category = category || workout.category;
    workout.exercises = exercises || workout.exercises;
    workout.notes = notes || workout.notes;
    // Fix: Assign tags to the workout object
    workout.tags = tags || workout.tags;

    const updatedWorkout = await workout.save();
    res.json(updatedWorkout);
  } else {
    res.status(404).json({ message: 'Workout not found' });
  }
};
```

Getting user's Nutrition data from database in Nutrition log, saving as history

```
// Nutrition logs
const getNutrition = async (req, res) => {
  try {
    const logs = await Nutrition.find({ user: req.user._id }).sort({ createdAt: -1 });
    res.status(200).json(logs);
  } catch (error) {
    res.status(500).json({ message: 'Server Error' });
  }
};
```

CODE SNIPPETS

Setting fields in databse to store user input for Nutritions

```
const addCalorieLog = async (req, res) => {
  try {
    const { foodName, calories, protein, carbs, fats } = req.body;
    const log = await Calorie.create({
      user: req.user._id,
      foodName,
      calories,
      protein,
      carbs,
      fats
    });
    res.status(201).json(log);
  } catch (error) {
    res.status(400).json({ message: 'Error saving calorie log' });
  }
};
```

CODE SNIPPETS

Routes:

Workout Api routes

backend > routes > JS workoutRoutes.js > ...

```
const express = require('express');
const router = express.Router();
const { createWorkout, getWorkouts, deleteWorkout,
  updateWorkout, toggleExercise } = require('../controllers/workoutController');
const { protect } = require('../middleware/authMiddleware');

// 'protect' ensures only logged-in users can create workouts
router.route('/')
  .post(protect, createWorkout)
  .get(protect, getWorkouts);


router.route('/:id')
  .put(protect, updateWorkout)
  .delete(protect, deleteWorkout);

router.put('/:workoutId/exercises/:exerciseId', protect, toggleExercise);

module.exports = router;
```

Nutrition Api Routes

backend > routes > JS nutritionRoutes.js > ...

```
const express = require('express');
const router = express.Router();
const { addNutrition, getNutrition, clearNutrition } = require('../controllers/nutritionController');
const { protect } = require('../middleware/authMiddleware'); // Use your existing auth mid

router.route('/')
  .get(protect, getNutrition)
  .post(protect, addNutrition)
  .delete(protect, clearNutrition);

module.exports = router;
```

CONCLUSION

The Fitness Tracker application has been successfully developed as a full-stack web solution using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The system allows users to securely register, manage workout routines, track nutrition intake, and monitor their fitness progress in an organized and efficient manner. By integrating multiple modules into one centralized platform, the application provides a practical and user-friendly approach to personal health management.

The project demonstrates the implementation of core web development concepts such as authentication, CRUD operations, RESTful APIs, database schema design, and frontend-backend integration. The structured database design ensures efficient storage and retrieval of user data, while secure password encryption and protected routes maintain data privacy and security. The responsive user interface enhances usability across different devices.

Overall, the Fitness Tracker successfully achieves its objective of providing a scalable, secure, and efficient fitness management system. This project not only fulfills functional and non-functional requirements but also strengthens practical understanding of real-world application development and system design, preparing for future professional challenges in software development.